A PROJECT REPORT ON

SignSense

Real-Time Sign Language Recognition using MediaPipe Holistic and LSTM

Submitted by

TEAM 16

Soham Bacchuwar (20233571)
Sri Ram Varun Dittakavi (20233574)
Shubham Daya Vadher (20233568)
Varun Kumar (20233581)

October 2026

ABSTRACT

Sign language is the primary way many deaf and hard-of-hearing people communicate, yet very few hearing people understand it. In this project we built SignSense, a system that watches a person through an ordinary laptop webcam and converts their signs into English words in real time.

Instead of training a network on raw video, we use Google's MediaPipe Holistic model to locate the body and hand landmarks in every frame. These landmarks are reduced to 258 numbers per frame, and a sequence of 30 frames is passed to a stacked LSTM network that learns how the hands and body move while performing a sign. We recorded our own dataset of 90 sequences covering three signs: hello, thanks and I love you.

The model was trained for 700 epochs and correctly classified all 5 sequences in the held-out test set, giving an accuracy of 1.0. When run live on the webcam it was able to follow the signer from one sign to the next and display the recognised words on screen.

Keywords: sign language recognition, MediaPipe Holistic, LSTM, computer vision, deep learning

CONTENTS

1 Introduction..... 3

1.1 Problem Statement..... 3

1.2 Objectives..... 3

2 Background..... 4

3 Methodology..... 5

3.1 System Overview..... 5

3.2 Landmark Detection and Feature Extraction..... 5

3.3 Dataset..... 5

3.4 Data Preprocessing..... 6

3.5 Model Architecture..... 6

3.6 Training..... 7

3.7 Real-Time Recognition..... 7

4 Results and Discussion..... 8

5 Conclusion and Future Scope..... 10

References..... 11

1. Introduction

Communication is something most of us take for granted, but for people who rely on sign language even a simple conversation with a stranger can be difficult. Interpreters are not always available, and most hearing people have never learned to sign. A tool that can understand signs automatically and show their meaning as text would make everyday interactions much easier.

Recent progress in computer vision has made it possible to track the human body and hands accurately using just a normal webcam, without gloves or special sensors. This project uses that progress to build a working prototype of a sign-to-text translator that runs live on a laptop.

1.1 Problem Statement

To design and implement a system that captures a person performing sign-language gestures through a webcam, recognises which sign is being performed, and displays the corresponding word on screen in real time.

1.2 Objectives

- To detect body and hand landmarks from a live video feed using MediaPipe Holistic.
- To create our own dataset of sign sequences for the chosen signs.
- To train an LSTM-based deep learning model that classifies a sequence of frames into a sign.
- To evaluate the model using a confusion matrix and accuracy score.
- To run the trained model in real time and display the recognised words as a sentence.

The current version of the system recognises three signs: hello, thanks and I love you.

2. Background

MediaPipe Holistic. MediaPipe is an open-source framework from Google for building perception pipelines [1]. Its Holistic solution combines separate pose, face and hand models into one pipeline and returns 33 body-pose landmarks, 468 face landmarks and 21 landmarks for each hand from a single RGB image. Each landmark has normalised x and y coordinates and a relative depth z; pose landmarks also have a visibility score. Since the model runs in real time on a CPU, it is well suited to a webcam application.

Long Short-Term Memory (LSTM). A sign is not a single still pose but a movement that happens over time, so the model must be able to learn from sequences. LSTM networks, introduced by Hochreiter and Schmidhuber [3], are recurrent neural networks that process a sequence one step at a time while maintaining an internal memory. Gates inside each LSTM cell decide which information should be added to this memory, kept, or passed on, which lets the network learn patterns that span many time steps.

Why keypoints instead of images? Training a network directly on video frames would need a very large dataset and a lot of computing power. By first converting each frame to a small set of landmark coordinates, the input becomes much smaller and contains only the information that matters for signing: the position of the hands and body. This makes it possible to train an accurate model with a dataset that we could record ourselves.

3. Methodology

The entire project is implemented in a single Jupyter notebook using Python 3.11, OpenCV for webcam handling, MediaPipe for landmark detection, NumPy for storing data, scikit-learn for splitting and evaluating, and TensorFlow/Keras for the neural network.

3.1 System Overview

Fig. 1 shows how a single webcam frame moves through the system during live recognition. The same steps, up to keypoint extraction, are also used when collecting the training data, so the model always receives data in the same format.

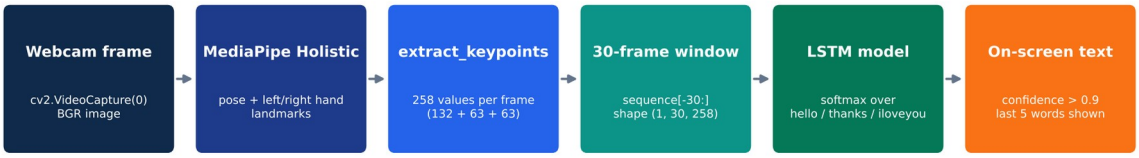


Fig. 1 Overall pipeline of SignSense

3.2 Landmark Detection and Feature Extraction

Each frame captured by OpenCV is first converted from BGR to RGB, since MediaPipe expects RGB images, and is then passed to the Holistic model with a minimum detection confidence and tracking confidence of 0.5. The detected pose and hand landmarks are drawn back onto the frame so the user can see what the system is tracking.

A function called extract_keypoints() converts the landmarks of one frame into a single feature vector. The pose contributes 33 × 4 = 132 values (x, y, z and visibility) and each hand contributes 21 × 3 = 63 values, giving a total of 258 values per frame. If a hand or the body is not detected in a frame, its part of the vector is filled with zeros, so the length always stays the same. We decided to leave out the 468 face landmarks (1,404 values) because the three signs depend mainly on the hands and arms, and including the face would have made the input more than six times larger.

3.3 Dataset

No ready-made dataset was used; we recorded our own data using the webcam. For each sign the program waits 5 seconds so the signer can get ready and then records 30 videos of 30 frames each. At the start of every video the message "STARTING COLLECTION" appears on screen and the program pauses for 2 seconds, giving a short break between repetitions. For every frame the 258-value keypoint vector is saved as a NumPy file in the folder MP_Data/<sign>/<video number>/<frame number>.npy.

Table 1 Summary of the dataset

Property	Value
Signs	hello, thanks, iloveyou
Videos (sequences) per sign	30
Frames per sequence	30
Values per frame	258
Total sequences	90
Total files	2,700

Fig. 2 shows one frame from each sign, plotted directly from the saved files. It is worth noting that the hello sign was performed with one hand in our recordings (the second hand appears in only 1.6% of its frames), while thanks and I love you involve both hands.

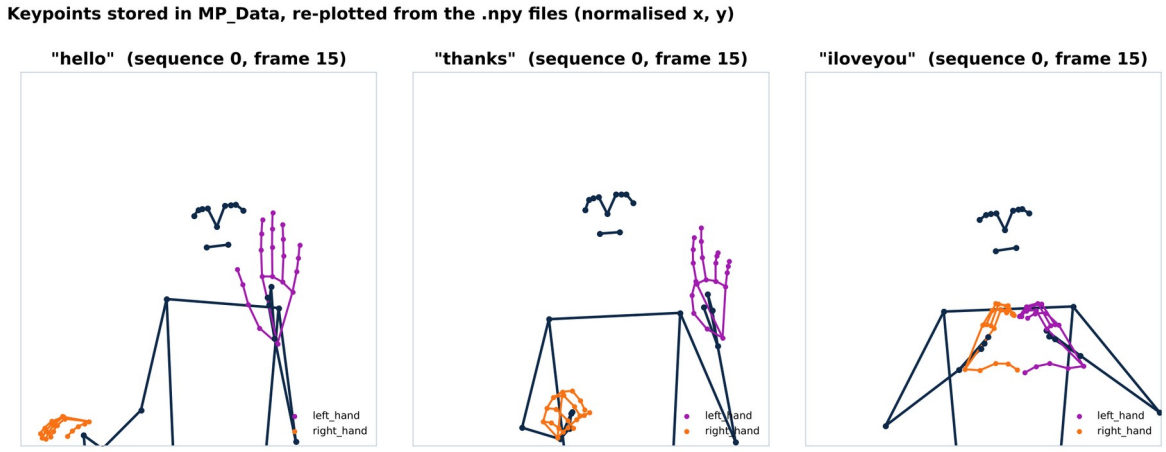


Fig. 2 Stored keypoints for one frame of each sign

3.4 Data Preprocessing

All 2,700 files are loaded back into memory, with the 30 frames of each video kept in order. This gives an input array X of shape (90, 30, 258). The sign names are mapped to numbers (hello = 0, thanks = 1, iloveyou = 2) and converted to one-hot vectors, so the labels y have shape (90, 3). The data is then split with train_test_split using a test size of 5%, which leaves 85 sequences for training and 5 for testing.

3.5 Model Architecture

The classifier is a sequential Keras model made up of three LSTM layers followed by three fully connected (dense) layers, as shown in Fig. 3 and Table 2. The first two LSTM layers return their output at every time step so that the next LSTM layer still receives a sequence, while the third LSTM layer returns only its final output, which summarises the whole movement. The dense layers then map this summary to the three classes, and the softmax activation turns the outputs into probabilities that add up to one. ReLU activation is used in all hidden layers.



Fig. 3 Architecture of the LSTM model

Table 2 Layer-wise summary of the model

Layer	Output Shape	Parameters
LSTM (64 units)	(30, 64)	82,688
LSTM (128 units)	(30, 128)	98,816
LSTM (64 units)	(64)	49,408
Dense (64, ReLU)	(64)	4,160
Dense (32, ReLU)	(32)	2,080
Dense (3, Softmax)	(3)	99
Total trainable parameters		237,251

3.6 Training

The model was compiled with the Adam optimiser and categorical cross-entropy loss, with categorical accuracy as the metric, and trained for 700 epochs using the default batch size of 32 (three batches per epoch). A TensorBoard callback was used to log the training progress. After training, the complete model was saved to the file action.keras.

3.7 Real-Time Recognition

For the live demo, the program keeps a list of the keypoint vectors from the most recent 30 frames. Every new frame is added to the end of the list and the oldest one is dropped, so the model always sees a sliding window of the latest movement. Once 30 frames are available, the window is passed to the model, which returns a probability for each sign.

A prediction is accepted only if its probability is higher than 0.9. To avoid writing the same word again and again while a sign is being held, a word is added to the sentence only if it is different from the previous word. The last five words are shown in a banner at the top of the video window, and the user can press q to close the program.

4. Results and Discussion

4.1 Training Performance

Fig. 4 shows how the training accuracy and loss changed over the 700 epochs. The model first reached 100% training accuracy at epoch 9. The loss was quite unstable during the early epochs and jumped sharply several times, the largest spike being 15.67 at epoch 37. After epoch 62 training settled down: the accuracy stayed at 100% for the rest of training and the loss kept falling steadily, ending at about 8.1×10^{-8}.

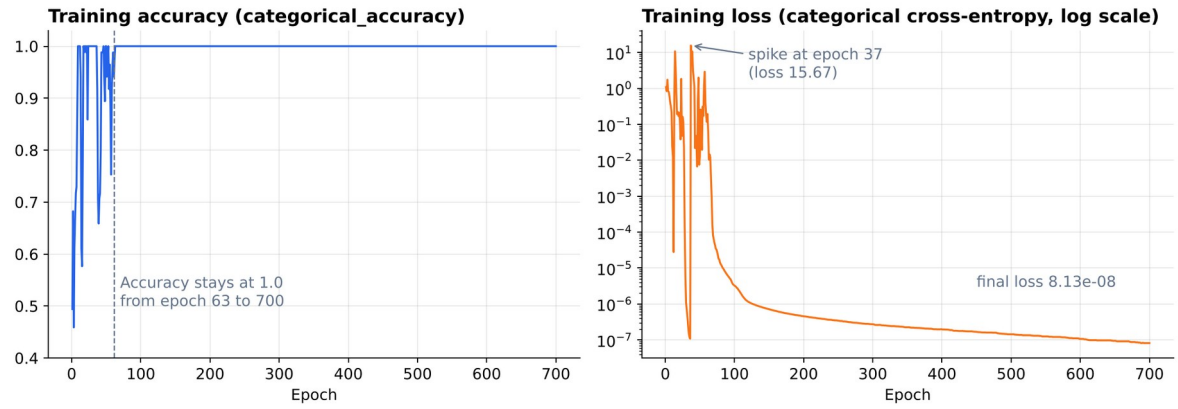


Fig. 4 Training accuracy and loss over 700 epochs

4.2 Evaluation on the Test Set

The trained model was tested on the 5 sequences that were kept aside (2 hello, 1 thanks and 2 I love you). We used scikit-learn's multilabel confusion matrix, which gives a separate true/false positive and negative count for each sign, along with the overall accuracy score. As Fig. 5 shows, every test sequence was classified correctly, with no false positives or false negatives for any of the signs, giving an accuracy of 1.0.

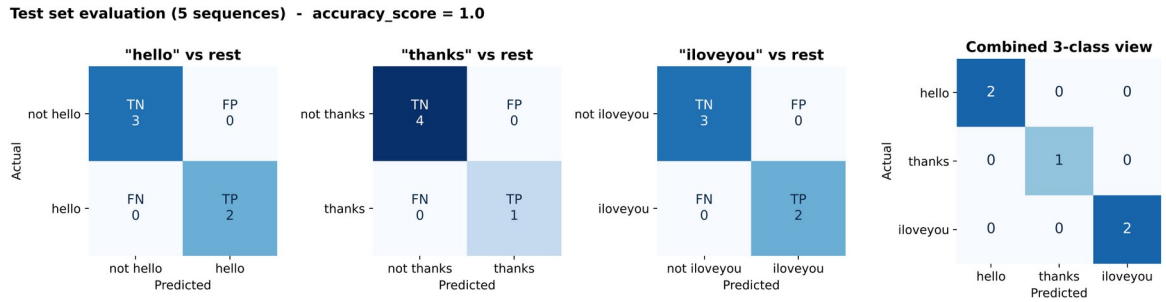


Fig. 5 Confusion matrices for the test set

4.3 Real-Time Testing

When tested live, the model's printed output followed the signer as they changed from one sign to another, moving through hello, thanks, I love you, thanks, hello and thanks, with each sign recognised consistently over many consecutive frames. Most predictions took around 15 to 40 milliseconds, which was fast enough for the system to feel smooth while running on a laptop.

While these results are very encouraging, they should be read with the size of the experiment in mind. The test set had only five sequences, and all the data was recorded by our own team, so a larger and more varied test would be needed to judge how well the model works for other people and in other settings.

5. Conclusion and Future Scope

In this project we built SignSense, a real-time sign language recognition system that needs nothing more than a webcam. By combining MediaPipe Holistic for landmark detection with a stacked LSTM network for sequence classification, the system recognised the signs hello, thanks and I love you with full accuracy on our test data and worked smoothly in live use. Using keypoints rather than raw images kept the model small and allowed us to train it on a dataset that we recorded ourselves.

There are several ways the project can be extended in the future:

- Adding more signs, so that the system can be used for real conversations.
- Recording data from more people, with different backgrounds and lighting, to make the model more robust.
- Using a larger test set or k-fold cross-validation for a more reliable measure of accuracy.
- Adding text-to-speech so that the recognised words can be spoken aloud.

References

[1] C. Lugaresi et al., "MediaPipe: A Framework for Building Perception Pipelines," arXiv:1906.08172, 2019.

[2] Google, "MediaPipe Holistic," MediaPipe Solutions documentation. Available: https://github.com/google-ai-edge/mediapipe

[3] S. Hochreiter and J. Schmidhuber, "Long Short-Term Memory," Neural Computation, vol. 9, no. 8, pp. 1735-1780, 1997.

[4] M. Abadi et al., "TensorFlow: A System for Large-Scale Machine Learning," in Proc. OSDI, 2016.

[5] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," Journal of Machine Learning Research, vol. 12, pp. 2825-2830, 2011.

[6] G. Bradski, "The OpenCV Library," Dr. Dobb's Journal of Software Tools, 2000.

[7] Project repository: https://github.com/soham200521/TEAM_16